→ Views em SQL

- É uma tabela derivada de outras tabelas
- A derivação a partir de tabelas de base (física) ou outras views (virtuais)
- Útil em consultas com bastante frequência
- Operações em SQL
  - Atualização: única tabela (atualizável), várias tabelas (as vezes atualizável), funções de agregação (não são atualizáveis)

```
CREATE VIEW <nome_visão> AS
<Consulta SQL>
```

Empregado (nominic, nomint, nomefim, numat, dtnasc, ender, sexo, salario, nsupervisor, ndepto)
Trabalha Em (numemp, nproj, horas)
Projeto (nomeproj, numproj, locproj, ndep)

Projetos alocados aos empregados

```
CREATE VIEW V_PROJ_EMPR AS
SELECT *
FROM Empregado, Trabalha Em, Projeto
WHERE numat = numemp AND nproj = numproj;
```

Empregado (nominic, nomint, nomefim, numat, dtnasc, ender, sexo, salario, nsupervisor, ndepto)
Trabalha Em (numemp, nproj, horas)
Projeto (nomeproj, numproj, locproj, ndep)

Projetos alocados aos empregados

```
CREATE VIEW V_PROJ_EMPR(nominic, nomeproj, horas) AS
SELECT nominic, nomeproj, horas
FROM Empregado, Trabalha Em, Projeto
WHERE numat = numemp AND nproj = numproj;
```

→ Nome dos empregados e nome de seus supervisores diretos

Emp (empinic, empfim, nsup) ← π nominic, nomefim, n sup (Empreg)
Sup (supinic, supfim, nsup) ← π nominic, nomefim, numat (Empreg)

π empinic, empfim, supinic, supfim (Emp ⋈ Sup)

```
SELECT  einic, efim, sinic, sfim
FROM    Empregado (nominic, nomefim, nsuper) AS
        Emp (einic, efim, nsup) NATURAL JOIN
        Empregado (nominic, nomefim, numat) AS
        Sup (sinic, sfim, nsup);
```

→ Recuperar o nome do empregado e, se o empregado possuir supervisor, recuperar também o nome do seu supervisor imediato

```
SELECT  e.nominic as IniEmp,
        e.nomefim as FimEmp,
        s.nominic as IniSup,
        s.nomefim as FimSup
FROM    Empregado e LEFT OUTER JOIN
        Empregado s
        ON e.super = s.numat
```

→ Para cada projeto localizado em Dourados, selecionar o número do projeto, o número e o nome do chefe do departamento que controla o projeto

```
SELECT  numproj, ndep, nominic
FROM    PROJETO JOIN
        Departamento ON numdep = ndep
        JOIN Empregado ON nchefe = numat
WHERE   locproj = 'Dourados'
```

→ OUTER JOIN

- Uma junção em que as tuplas de uma das relações aparecem no resultado mesmo que não tenham par na outra relação (com os atributos da outra relação preenchidos com valores nulos)

- Left Outer Join: As tuplas da relação do lado esquerdo aparecem mesmo sem correlação.
- Right Outer Join: As tuplas da relação do lado direito aparecem mesmo sem correlação
- Full Outer Join: As tuplas de ambas relações aparecem mesmo sem correlação

→ LEFT OUTER JOIN

Buscar o nome de todos os empregados. Se o empregado possuir dependentes, selecionar o nome e sexo de cada um dos seus dependentes

```
SELECT  e.nominic, d.depend, d.sexo
FROM    Empregado AS e LEFT OUTER JOIN
        Dependente AS d ON e.numat = d.numemp
```

→ RIGHT OUTER JOIN

Buscar os departamentos e seus respectivos chefes, incluindo os departamentos que não tem chefe

```
SELECT  d.nomedep, e.nominic
FROM    Empregado AS e RIGHT OUTER JOIN
        Departamento AS d ON e.numat = d.nchefe
```

→ FULL OUTER JOIN

Buscar os departamentos e seus respectivos chefes, incluindo os departamentos que não tem chefe e os empregados que não chefiam nenhum departamento

```
SELECT  d.nomedep
FROM    Empregado AS e FULL OUTER JOIN
        Departamento AS d ON e.numat = d.nchefe
```

→ Junções

→ CROSS JOIN

- Simplesmente substitui a vírgula nas operações de junção (FROM). Se não houver condição de junção o resultado é produto cartesiano
- Relacione os nomes dos empregados e os nomes de seus dependentes

```
SELECT  e.nominic, d.depend
FROM    EmpregadoAS e CROSS JOIN Dependente AS d
WHERE   e.numat = d.numemp
```

→ NATURAL JOIN

- Serve para indicar uma junção natural, logo não é necessário indicar quais as colunas de junção. Elimina a duplicidade dos atributos de junção

Relacione o nome e o endereço dos empregados que trabalham no departamento de informática

```
SELECT  nominic, ender
FROM    Empregado NATURAL JOIN Departamento As Depto(nomedep, ndepto, nchefe, dtinichef)
WHERE   nomedep = 'Informática'
```

→ INNER JOIN ou JOIN

- Forma mais comum de junção em que são especificadas as condições de junção usando a cláusula ON. Retorna as tuplas que satisfaçam as condições de junção
- Relacione os nomes dos empregados e os nomes de seus dependentes

```
SELECT  nominic, depend
FROM    Empregado INNER JOIN Dependente ON numat = numemp
```

OR

JOIN

→ Número de salários iguais dos empregados

$\gamma_{sal, COUNT(*) \rightarrow d_sal}$ (Emprog)

```
SELECT    sal, COUNT(*)
FROM      Emprog
GROUP BY  sal
```

→ A média dos salários e o número de empregados para cada departamento

```
SELECT  rdepto, COUNT(*), AVG(Salario)
FROM    Empregado
GROUP BY rdepto;
```

→ Para cada projeto o seu número, o seu número, o seu nome e a quantidade de empregados que trabalham no projeto

```
SELECT  numproj, nomeproj, COUNT(*)
FROM    TrabalhaEm, Projeto
WHERE   nproj = numproj
GROUP BY numproj, nomeproj;
```

→ Empregados que possuem mais de 2 dependentes

```
SELECT *
FROM   Empregado
WHERE  (SELECT COUNT(*)
        FROM   Dependente
        WHERE  numat = numemp)
>2;
```

→ Para cada projeto com mais de 5 empregados, buscar o número do projeto, o nome do projeto e o número de empregados que trabalham nele

```
SELECT  numproj, nomeproj, COUNT(*)
FROM    Projeto, TrabalhaEm
WHERE   numproj = nproj
GROUP BY numproj, nomeproj
HAVING COUNT(*)>5;
```

→ Média dos salários dos empregados do Departamento de Pesquisa

```
SELECT    AVG(salario)
FROM      Empregado, Departamento
WHERE     nomedep = 'Pesquisa' AND numdep = ndepto
```

→ Número de empregados do Departamento de Pesquisa

```
SELECT    COUNT(*)
FROM      Empregado, Departamento
WHERE     nomedep = 'Pesquisa' and numdep = ndepto;
```

→ Número de diferentes salários da empresa

```
SELECT    COUNT(DISTINCT salario)
FROM      Empregado;
```

→ Agrupamento

- Em alguns casos necessitamos aplicar a função agregada em subgrupos de linhas

→ Média dos salários dos empregados por departamento

$\gamma_{ndepto, AVG(sal) \rightarrow avg_sal}(Empreg)$

```
SELECT    ndepto, AVG(sal)
FROM      Empreg
GROUP BY  ndepto;
```

→ Maior salário de um empregado por departamento

$\gamma_{ndepto, MAX(sal) \rightarrow max_sal}(Empreg)$

```
SELECT    ndepto, MAX(sal)
FROM      Empreg
GROUP BY  ndepto
```

→ Empregados que trabalham em apenas um projeto

```
SELECT *
FROM Empregado AS e
WHERE UNIQUE(
        SELECT t. numemp
        FROM TrabalhaEm as t
        WHERE e. numat = t. numemp
);
```

→ Funções Agregadas

- SQL possui as seguintes funções agregadas: SUM, COUNT, MIN, MAX e AVG

→ Count: Conta o número de tuplas resultante da consulta

→ SUM, MIN, MAX, AVG: aplicadas sobre um conjunto de valores

→ Soma de todos os salários

$\gamma_{SUM(sal) \rightarrow sum\text{-}sal}(Empreg)$

```
SELECT  SUM(sal)
FROM    Empreg
```

→ Número de Empregados

$\gamma_{COUNT(*) \rightarrow ct\text{-}emp}(Empreg)$

```
SELECT  COUNT(*)
FROM    Empreg
```

→ Maior salário entre os Empregados

$\gamma_{MAX(sal) \rightarrow max\text{-}sal}(Empreg)$

```
SELECT  MAX(sal)
FROM    Empreg
```

→ Considerando todos os empregados, selecionar a soma de todos os salários, o maior, menor e a média dos salários

```
SELECT  SUM(salario),
        MAX(salario),
        MIN(salario),
        AVG(salario)
FROM    Empregado;
```


Departamento As d ON e. numat = d. nchefe

→ Nomes dos empregados que não têm dependentes

$\pi_{nominic, nomefim}(Empreg) - \pi_{nominic, nomefim}(Empreg \bowtie_{numat = numemp} Depen)$

```
SELECT nominic, nomefim
FROM   Empregado
WHERE  NOT EXISTS (
       SELECT *
       FROM   Dependente
       WHERE  numat = numemp
       );
);
```

→ Nome dos empregados que trabalham em projetos controlados pelo mesmo departamento em que estão lotados

```
SELECT   e.nominic, e.nomint, e.nomefim
FROM     Empregado AS e
WHERE    EXISTS (
         SELECT p.*
         FROM   Projeto AS p, TrabalhaEm AS t
         WHERE  p.nrdep = e.nrdepto AND t.numemp = e.numat AND t.nrproj = p.numproj
);
```

→ A função UNIQUE(Q)

- Função booleana
- Parâmetro: consulta em SQL
- Retorna verdadeiro se duplicidade de tuplas não são encontradas como resultado da consulta Q

→ Exists

- Quantificador existencial do cálculo relacional
  - Testa se um predicado V ou F na subconsulta
  - Para cada tupla da consulta externa a ser analisada, a subconsulta é executada (consulta correlacionada)
  - Caso Exists retornar verdadeiro - não vazia. Se Exists retornar falso - vazia

```
select lista_atributos
from   tabela1 [, ...]
where  [NOT] EXISTS (consulta_SQL)
```

- Relacione os nomes dos clientes que estão em atraso na devolução das fitas que locaram

```
SELECT   nomecli
FROM     Cliente
WHERE    EXISTS (
         SELECT *
         FROM   Locacao
         WHERE  idcli = Cliente.idcli
         AND    dtdevalprev < SYS_DATE()
         AND    dtdevalreal IS NULL
);
```

→ Nomes dos chefes de departamento que têm pelo menos um dependente

$\pi_{nominc,\ nomefun}(Dep \bowtie_{nchefe = numat} (Empreg \bowtie_{numat = numemp} Depen))$

```
SELECT   nominc
FROM     Empregado
WHERE    EXISTS (
         SELECT *
         FROM   Dependente
         WHERE  numat = numemp)
AND EXISTS (
         SELECT *
         FROM   Departamento
         WHERE  numat = nchefe);
```

→ Subconsultas - ALL

- Para comparação de elemento - conjunto
- Testa se um valor é >, <, =, ... que todos valores de um conjunto

```
select   lista_atributos
from     tabela [, ...]
where    atributo_ou_expressão [=|<|>|..] ALL (consulta-SQL)
```

- Encontrar nomes dos clientes com data de admissão máxima

```
SELECT   nomecli
FROM     Cliente
WHERE    dtadm >= ALL (
         SELECT dtadm
         FROM   Cliente
);
```

→ Nome dos empregados cujo salário é maior do que o salário de todos os empregados do departamento 5

```
SELECT   nominic, nomint, nomfim
FROM     Empregado
WHERE    salario > ALL (
         SELECT salario
         FROM   Empregado
         WHERE  nodepto = 5
);
```

→ Qual o resultado do seguinte SELECT?

```
SELECT DISTINCT numemp
FROM   TrabalhaEm
WHERE  (nproj, horas) IN (
       SELECT nproj, horas
       FROM   TrabalhaEm
       WHERE  numemp = '1234'

);
```

→ Subconsultas - ANY

- Para comparação de elemento - conjunto
- Testa se um valor é >, <, =, ... que <u>algum</u> valor de um conjunto

```
select lista - atributos
from tabela1 [, ...]
where atributo - ou - expressao [=|<|<=|>|>=|<>|!=] ANY
                                                (consulta_sql)
```

- Encontrar nomes dos empregados cuja data de nascimento é posterior à de algum empregado do departamento 5

```
SELECT nomeinic, nomeint, nomefim
FROM   Empregado
WHERE  dt_nasc > ANY (
       SELECT dtnasc
       FROM   Empregado
       WHERE  ndepto = 5

);
```

→ O operador IN

- Compara um valor de v com um conjunto (ou multiconjunto) de valores V e avalia como verdadeiro se v for um dos elementos de V. Caso contrário, avalia falso.
- Para comparar elemento - conjunto. Devem ser compatíveis

```
select lista_atributos
from   tabela1 [...]
where  atributo_ou_expressão [NOT] IN (consulta_SQL)
```

→ Subconsultas - IN

- Relacione os códigos das locações e os nomes dos clientes que as fizeram para contas de locação que estão vencendo hoje

```
SELECT idloc, nomecli
FROM   Cliente NATURAL JOIN Locacao
WHERE  idct IN(
          SELECT idct
          FROM   Conta
          WHERE  dtvenc = SYS_DATE()
);
```

→ Nomes dos empregados que não têm dependente

- $\pi_{nominic,\ nomefim}(Empreg) - \pi_{nominic,\ nomefim}(Empreg \bowtie_{numat = numemp} Depen)$

```
SELECT nominic, nomefim
FROM   Empregado
WHERE  numat not IN(
          SELECT numemp
          FROM   Dependente
);
```

→ Consultas aninhadas ou subconsultas

- Forma alternativa de especificar consultas envolvendo relacionamentos entre tabelas
- Otimização
  - Filtragens prévias de dados na subconsulta
    - ↳ Apenas tuplas/atributos de interesse são combinados com dados da(s) tabela(s) da consulta externa
- Cláusulas

- nome_atributo [NOT] IN (consulta_SQL)
- nome_atributo [<|<=|>|>=|<>|!=] ANY (consulta_SQL)
- nome_atributo [<|<=|>|>=|<>|!=] ALL (consulta_SQL)
- nome_atributo [NOT] EXISTS (consulta_SQL)

→ Subconsultas que produzem valor escalar

- A subconsulta produz apenas um elemento como resultado, para comparação de elemento-elemento

```
SELECT  lista_atributos
FROM    tabela1 [, ...]
WHERE   atributo ou expressão [=|<|<=|>|>=|<>|!=] (consulta_SQL com um único elemento)
```

- Relacione os funcionários que tem salário maior que o funcionário de CPF 999.999.999-99

```
SELECT  nome
FROM    Funcionario
WHERE   salario > (
        SELECT  Salario
        FROM    Funcionario
        WHERE   cpf = 999999999999
);
```

→ Comparação de substrings

• O operador LIKE permite a comparação com parte de uma string usando coringas "%" e "_"

SELECT * FROM Empregado WHERE ender LIKE "R _ X"

→ R X

SELECT * FROM Empregado WHERE ender LIKE "% da Silva"

→ Antonieta da Silva, da Silva

SELECT * FROM Empregado WHERE ender LIKE "C%"

→ Ceará, Comendador

SELECT * FROM Empregado WHERE ender LIKE "% da %"

→ Manoel da Costa, da Costa e Silva, R Hilda

SELECT * FROM Empregado WHERE ender LIKE "Star ----"

→ Star Wars, Star Trek

SELECT * FROM Empregado WHERE ender LIKE "Star ---- %"

→ Star Wars 2018

SELECT * FROM Empregado WHERE ender LIKE "%rLy %" || "%iLL%"

→ Beverly Hills Cop, Praia de Beverly Hills

→ Operadores de conjunto - UNION (união), EXCEPT (diferença), INTERSECT (interseção)

• Primeiro nome e número do objeto que tenha um Silva como último nome que trabalha no projeto ou é chefe do departamento que controla o projeto"

$Prj(nproj, numdep) \leftarrow \pi_{numproj, ndep}(Proj)$

$\pi_{nominic, nproj}(\sigma_{nomefim = 'Silva'}(Empreg) \bowtie_{numat = numemp} Trab) \cup$

$\pi_{nominic, nproj}(Prj \bowtie_{Dep} \bowtie_{nchefe = numat} (\sigma_{nomefim = 'Silva'}(Empreg))) \Leftrightarrow$

```
(SELECT nominic, nproj AS numproj
FROM TrabalhaEm, Empregado
WHERE numemp = numat AND nomefim = 'Silva')
UNION
(SELECT nominic, num proj
FROM Projeto, Departamento, Empregado
WHERE ndep = numdep AND nchefe = numat
AND nomefim = 'Silva');
```

→ Cláusula WHERE não especificada

```
SELECT   nominic, nomefim, depend
FROM     Empregado, Dependente
```

- Combina cada nome de Empregado com todos nomes de dependentes, sem relação de qualificação, no resultado da consulta: Produto Cartesiano

→ Sintaxe do comando SELECT

```
SELECT <lista de campos>
FROM <lista de tabelas>
[WHERE <condição>]
[GROUP BY <campos que definem o grupo>
[HAVING <condição sobre a função de grupo>]]
[ORDER BY <campos do resultado da consulta>]
```

→ Ordenando o resultado: ASC / DESC

```
SELECT   nominic, salario
FROM     Empregado
WHERE    ndepto = 5 OR sexo = 'F';
ORDER BY ~~nomin~~ salario, nominic
```

```
SELECT   nominic
FROM     Empregado
ORDER BY nominic DESC
```

```
SELECT nominic
FROM   Empregado
ORDER BY nominic ASC
```

```
SELECT   salario, nominic
FROM     Empregado
ORDER BY salario DESC, nominic ASC
```

→ Nomes de atributos ambíguos (Alias - apelidos)

i. O nome do empregado, o nome do dependente e o sexo do dependente para empregados do sexo masculino ⟺

```
SELECT  E.nominic, E.nomefim, D.depend, D.sexo
From    Empregado AS E, Dependente AS D
WHERE   E.numat = D.numemp AND E.sexo = 'M'
```

→ Nomes dos empregados e nomes de seus supervisores diretos

$E \leftarrow \pi_{nominic,\ nomefim,\ numat,\ nsup}(Empreg)$

$S \leftarrow \pi_{nominic,\ nomefim,\ numat,\ nsup}(Empreg)$

$\pi_{E.nominic,\ E.nomefim,\ S.nominic,\ S.nomefim}(E \bowtie_{E.nsup = S.numat} S)$ ⟺

```
SELECT  E.nominic, E.nomefim, S.nominic, S.nomefim
FROM    Empregado AS E, Empregado AS S
WHERE   E.nsup = S.numat;
```

→ Conjuntos explícitos

```
SELECT  DISTINCT numemp
FROM    TrabalhaEm
WHERE   nproj IN (1,2,3);
```

→ Consultar a valores NULL

```
SELECT  nominic, nomefim
FROM    Empregado
WHERE   nsupser IS NULL;
```

→ Nome e salário dos empregados que estão lotados no Depto de Pesquisa

$\pi_{nominic, nomint, nomefim, salario}(\sigma_{nomedep = 'Pesquisa'}(Empregado \bowtie_{ndepto = numdep} Departamento))$ <=>

```
SELECT  nominic, nomint, nomefim, salario
FROM    Empregado, Departamento
WHERE   ndepto = numdep AND nomedep = 'Pesquisa';
```

→ Para cada projeto locado em Terena, listar o número do projeto, número do departamento controlado e último nome do chefe

$\pi_{numproj, ndep, nomefim}(\sigma_{locproj = 'Terena'}(Projeto \bowtie_{ndep = numdep} Departamento \bowtie_{nchefe = numat} Empregado))$ <=>

```
SELECT  numproj, ndep, nomefim
FROM    Projeto, Departamento, Empregado
WHERE   ndep = numdep AND nchefe = numat
        AND locproj = 'Terena';
```

→ Nomes dos empregados que trabalham nos projetos controlados pelo depto 5

$\pi_{nominic, nomefim}(\sigma_{ndep = 5}(Emp \bowtie_{numat = numemp} TrabalhaEm \bowtie_{nproj = numproj} Proj))$ <=>

```
SELECT  nominic, nomint, nomefim
FROM    Empregado, TrabalhaEm, Projeto
WHERE   numat = numemp AND nproj = numproj
        AND ndep = 5
```

→ Nomes de atributos ambíguos (Qualificação)

. O nome do empregado, o nome do dependente ~~e~~ e o sexo do dependente para empregados do sexo masculino

```
SELECT  nominic, nomefim, depend, Dependente.sexo
FROM    Empregado, Dependente
WHERE   numat = numemp AND Empregado.sexo = 'M'
```

→ Matrícula e salário dos empregados do departamento 5 que recebem até 300

$\pi_{numat, salario}(\sigma_{ndepto = 5 \wedge salario <= 300}(Empregado))$ ⇒

```
SELECT numat, salario
FROM   Empregado
WHERE  ndepto = 5 AND salario <= 300;
```

→ Matrícula e salário dos empregados do departamento 5 ou do sexo feminino

$\pi_{numat, salario}(\sigma_{ndepto = 5 \vee sexo = 'F'}(Empregado))$

```
SELECT numat, salario
FROM   Empregado
WHERE  ndepto = 5 OR sexo = 'F';
```

→ Faça a junção (de igualdade) de projetos e departamentos que controlam estes projetos

$Projeto \bowtie_{ndep = numdep} Departamento$ ⇒

```
SELECT *
FROM   Projeto, Departamento
WHERE  ndep = numdep;
```

→ O que fazer para obter informações sobre empregados e sobre os projetos onde trabalham?

. Empregado (nominic, nomint, nomefim, numat, dtnasc, ender, sexo, salario, nsupervisor, ndepto)
. TrabalhaEm (numemp, nproj, horas)
. Projeto (nomeproj, numproj, locproj, ndep)

$Empregado \bowtie_{numat = numemp} TrabalhaEm \bowtie_{nproj = numproj} Projeto$ ⇔

```
SELECT *
FROM   Empregado, TrabalhaEm Projeto
WHERE  numat = numemp AND nproj = numproj
```

→ Comando SELECT Básico

SELECT <lista de atributos> → nomes dos atributos cujos valores devem ser recuperados pela consulta

FROM <lista de tabelas> → nomes das tabelas exigidas para o processamento da consulta

WHERE <condição> → expressão condicional que identifica que registros serão recuperados pela consulta

→ Empregados que trabalham no departamento 5

$\sigma_{ndepto=5}$ (Empregado) ⇒

```
SELECT *
FROM   Empregado
WHERE  ndepto = 5;
```

→ Empregados do sexo feminino que trabalham no depto. 5

$\sigma_{sexo='F' \wedge ndepto=5}$ (Empregado) ⇒

```
SELECT *
FROM   Empregado
WHERE  sexo = 'F' AND ndepto = 5;
```

→ Nome, número de matrícula e salário dos empregados

$\pi_{nominic, nomefim, numat, salario}$ (Empregado) ⇒

```
SELECT nominic, nomefim, numat, salario
FROM   Empregado;
```

→ Lista de salário dos empregados

$\pi_{salario}$ (Empregado) ⇒

```
SELECT salario
FROM   Empregado;
```

ou

```
SELECT ALL salario
FROM   Empregado
```

* Os salários aparecem mesmo se repetidos, com ou sem a cláusula ALL

```
SELECT DISTINCT salario
FROM   Empregado;
```

- Caso a cláusula DISTINCT vier acompanhada de mais de um atributo, DISTINCT aplica-se a cada registro como um todo, isto é, serão mantidos apenas os registros não repetidos para o grupo de atributos envolvidos

→ Comando UPDATE

SC · Para modificar valores de atributos de uma ou mais tuplas selecionadas, cláusula
FF SET adicional especifica atributos que serão modificados e seus valores.
WI

```
UPDATE    Projeto
SET       loc_proj = 'Teresina', ndep = 5
WHERE     num_proj = 10;
```

. Várias tuplas:

```
UPDATE    Empregado
SET       salario = salario * 1.1
WHERE     ndepto IN (SELECT  numdep
                     FROM    Departamento
                     WHERE   nomedep = 'Pesquisa';
```

. Cada comando de atualização só se aplica a uma tabela

→ Comando SELECT

. Comando básico para recuperação de informação

. Comando SELECT do SQL não corresponde à operação de seleção da álgebra relacional

. Modelo Relacional x SQL:

. No modelo relacional as entidades são tratadas como um conjunto, sem os elementos com os mesmos valores

. SQL trata uma tabela não como um conjunto, logo registros duplicados podem aparecer no resultado da consulta

. Para restringir tabelas a conjuntos: DISTINCT

· Segunda forma: especificar nomes de atributos explícitos que correspondam aos valores no comando
· Os outros são preenchidos com NULL

```
INSERT INTO Empregado(
    nominic, nomefim, numat, ndepto)
    VALUES('Roberto', 'Gomes', 9802, 4);
```

```
INSERT INTO Empregado(
    nominic, nomefim, ndepto)
    Values('Mário', 'Gomes', 2);
```

↓

Rejeita pois faltou especificar Numat, por ser NOT NULL
- Depto 2 não existe.

→ → Tabelas Temporárias

```
CREATE TABLE Info-Depto
    (nomedepto VARCHAR(15),
     numempgs  INTEGER,
     saltot    INTEGER);
```

→ ...

π_n INSERT INTO Info-Depto (nomedepto, numempgs, saltot)

```
        SELECT   nomedep, COUNT(*), SUM (salario)
→ L     FROM     Departamento, Empregado
        WHERE    numdep = ndepto
π sal   GROUP BY nomedep;
```

* Os ... // Consultas sobre Info-Depto

SELE DROP TABLE Info-Depto /* ao final do serviço*/

FRO → Comando DELETE

```
· Cas DELETE FROM Empregado
DIST/ WHERE       numat = '1234';
```

```
DELETE FROM Empregado
WHERE       ndepto IN
            (SELECT   numdepto
            FROM      Departamento
            WHERE     nomedep = 'Pesquisa');
```

```
apenas DELETE FROM Empregado
       WHERE       nomefim = 'Paulo';
```

```
DELETE FROM Empregado;
```

. ALTER TABLE Empregado ALTER nchefe SET DEFAULT '3034';
↳ Define uma nova cláusula Default
. ALTER TABLE Empregado MODIFY nome_cargo VARCHAR(30);
↳ Altera o tamanho de um atributo
. ALTER TABLE Empregado RENAME TO Empregados;
↳ Altera o nome de uma tabela
. Para eliminar atributo:
. ALTER TABLE Empregado DROP endereco CASCADE;
↳ ~~Não permi~~ Exclui também os atributos filhos
. ALTER TABLE Empregado DROP endereco RESTRICT;
↳ Não permite a exclusão se houver atributo filho
. Para alterar restrições:
. ALTER TABLE Empregado DROP CONSTRAINT emprsuper CASCADE;
↳ A exclusão da restrição somente é permitida se houver um nome dado anteriormente à ela
. ALTER TABLE Empregado ADD CONSTRAINT ...;

→ DROP SCHEMA e DROP TABLE
. DROP SCHEMA Empresa CASCADE;
↳ Remove um esquema;
. DROP TABLE Dependente CASCADE;
↳ Remove a tabela e sua definição
. Cascade: Remove tudo relativo ao esquema/tabela
. Restrict: Remove se não houver nenhum elemento/restrição

↳ DML - SQL2
→ Comando INSERT
. Acrescenta um registro na tabela
. Atributos na mesma ordem que em CREATE TABLE

INSERT INTO Empregado VALUES (
'Ricardo', 'K', 'Marini', 65 32, 30.12.82, 'Camo 412', 3140, '9886', 4),

→ ALTER TABLE

. Para alterar a estrutura de uma tabela, incluindo ou redefinindo um atributo ou restrição de uma tabela

. Havendo registros quando um atributo for acrescentado à tabela, o novo atributo será inicialmente nulo para todos os registros existentes

. A alteração do tipo ou tamanho do atributo terá efeito somente para os novos registros

→ ALTER TABLE - ADD

ALTER TABLE [esquema.] tabela
ADD (atributo tipo de dado [restrições], ...);

. esquema: Se omitido, será criado no esquema corrente
. tabela: nome da tabela
. ADD: adicionar atributos ou restrições à tabela
. atributo: nome do atributo que será acrescentado
. tipo de dado: tipo e tamanho do atributo que será acrescentado
. restrições: especificação das restrições para o atributo acrescentado

→ ALTER TABLE - DROP/ALTER

ALTER TABLE [esquema.] tabela
DROP (atributo especificações);

! ALTER TABLE [esquema.] tabela
* ALTER (atributo especificações)

. esquema: Se omitido a tabela será criada no esquema corrente
. tabela: nome da tabela
. ALTER: modificar atributos ou restrições à tabela
. DROP: excluir atributos ou restrições da tabela
. atributo: nome do atributo que será alterado
. especificações: tipo de dado, tamanho de atributo, restrições, etc

→ ALTER TABLE

. Para adicionar atributo:
. ALTER TABLE Empregado ADD cargo VARCHAR(12);
. Para fazer alteração:
. ALTER TABLE Empregado ALTER COLUMN cargo TO nome_cargo;
↳ Altera o nome de um atributo
. ALTER TABLE Empregado ALTER nchefe DROP DEFAULT
↳ Remove uma cláusula default existente

→ Ações de gatilho referencial ON UPDATE

. ON UPDATE {NO ACTION, RESTRICT, CASCADE, SET NULL, SET DEFAULT}

. NO ACTION é o padrão quando ON UPDATE não está especificado. Ao alterar a chave primária do registro pai com registros filhos, a atualização falhará.

. RESTRICT: nenhuma mudança na tabela pai.

. CASCADE: ao alterar a chave primária do registro pai todos os registros filhos nas chaves estrangeiras serão alterados.

. SET NULL: ao alterar a chave primária do registro pai as chaves estrangeiras dos registros filhos receberão o valor NULL caso aceitável.

. SET DEFAULT: o mesmo acima porém valor Default.

→ CREATE TABLE - Departamento / Local Depto

```
CREATE TABLE Departamento (
    ...
    nchefe   CHAR(4)   NOT NULL   DEFAULT '8886',
    ...
  CONSTRAINT depchave
    PRIMARY KEY (numdep),
  CONSTRAINT depsecchave
    UNIQUE (nomedep),
  CONSTRAINT depchefe
    FOREIGN KEY (nchefe) REFERENCES Empregado (numat)
      ON DELETE  SET DEFAULT
      ON UPDATE CASCADE
);

CREATE TABLE Local Depto (
    ...
    PRIMARY KEY (numdep, locdep),
    FOREIGN KEY (numdep) REFERENCES Departamento (numdep)
      ON DELETE  CASCADE
      ON UPDATE  CASCADE
);
```

→ CREATE TABLE - Empregado - com qualificação

```
CREATE TABLE Empregado(
    ...
    ndepto      INT     NOT NULL     DEFAULT 1,
  CONSTRAINT empchave
    PRIMARY KEY (numat),
  CONSTRAINT empsupser
    FOREIGN KEY (nsuper) REFERENCES Empregado (numat)
            ON DELETE SET NULL
              ON UPDATE CASCADE,
  CONSTRAINT empdepto
    FOREIGN KEY (ndepto) REFERENCES Departamento (numdep)
            ON DELETE SET DEFAULT
              ON UPDATE CASCADE

);
```

. CONSTRAINT empchave: nome da restrição para chave primária dentro do esquema. Se não for informado será usado o nome do atributo numat.

→ Ações de gatilho referencial ON DELETE

ON DELETE {NO ACTION, RESTRICT, CASCADE, SET NULL, SET DEFAULT}

. NO ACTION: é o padrão quando ON DELETE não está especificado. Ao excluir um registro pai e registros filhos ficarão órfãos, o registro deletado será revertido

. RESTRICT: é o mesmo que NO ACTION, porém a exclusão será impedida caso resulte em filhos órfãos

. CASCADE: ao excluir um registro pai todos os registros filhos órfãos serão também excluídos

. SET NULL: ao excluir um registro pai as chaves estrangeiras dos registros filhos órfãos receberão o valor NULL quando possível

. SET DEFAULT: ao excluir um registro pai as chaves estrangeiras dos registros filhos órfãos receberão o valor DEFAULT quando possível

→ CREATE TABLE - TrabalhaEm

```
CREATE TABLE TrabalhaEm (
    numemp      CHAR(4)        NOT NULL,
    nproj       INT            NOT NULL,
    horas       DECIMAL(3,1)   NOT NULL,
  PRIMARY KEY (numemp, nproj),
  FOREIGN KEY (numemp) REFERENCES Empregado (numat),
  FOREIGN KEY (nproj) REFERENCES Projeto (numproj)
);
```

→ CREATE TABLE - Dependente

```
CREATE TABLE Dependente (
    numemp      CHAR(4)        NOT NULL,
    depend      VARCHAR(15)    NOT NULL,
    sexo        CHAR(1)
    dtnasc      DATE,
    tipo        VARCHAR(8),
  PRIMARY KEY (numemp, depend),
  FOREIGN KEY (numemp) REFERENCES Empregado (numat)
);
```

→ Pode ser Também:

. CREATE TABLE Empresa. Empregado

. Uma outra forma é apresentada dando nome às restrições de integridade, com o estabelecimento de restrições adicionais: as qualificações

→ Create Table - Departamento

```
CREATE TABLE Departamento {
    nomedep     VARCHAR(15)   NOT NULL,
    numdep      INT           NOT NULL,
    nchefe      CHAR(4),
    dtinichef   DATE.
PRIMARY KEY (numdep),
UNIQUE      (nome dep),
FOREIGN KEY (nchefe) REFERENCES Empregado (numat)
};
```

· Unique como restricão de tabela, podendo permitir a combinação de atributos com valores diferentes. Unique diferencia-se de PRIMARY KEY por permitir valores nulos

→ CREATE TABLE - LocalDepto

```
CREATE TABLE LocalDepto (
    numdep      INT         NOT NULL,
    locdep      VARCHAR(15) NOT NULL,
PRIMARY KEY (numdep, locdep),
FOREIGN KEY (numdep) REFERENCES Departamento (num dep)
```

→ CREATE TABLE - Projeto

```
CREATE TABLE Projeto (
    nomeproj    VARCHAR(20)   NOT NULL,
    numproj     INT           NOT NULL,
    locproj     VARCHAR(15),
    ndep        INT           NOT NULL,
PRIMARY KEY     numproj),
UNIQUE KEY      (nomeproj)
FOREIGN KEY     (ndep) REFERENCES Departamento (numdep)
);
```

Create Table [esquema.] tabela
(Nome do atributo tipo do dado [Default expr]
[restrição do atributo],
...,
[restrição da tabela]);

- esquema: Se omitido a tabela será criada no esquema corrente
- tabela: nome da tabela
- nome do atributo: onde o dado será armazenado
- tipo do dado: tipo do dado e tamanho do atributo
- DEFAULT expr: valor default quando dado inserido for omitido
- restrição do atributo: NOT NULL ou UNIQUE
- restrição da tabela: PRIMARY KEY, FOREIGN KEY

→ Create Table - Tipos de dados

- Numéricos:

↳ Int, Integer, SmallInt, Float, Real, Double Precision, Decimal (i, d), Numeric (i, d)

- Cadeia de caracteres:

↳ Char [Acter] (n), VarChar (n), Char [Acter] Varying (n)

- Cadeias de bits:

↳ Bit [Varying] (n)

- Datas:

↳ Date /* YYYY-MM-DD */, YEAR, MONTH, DAY

↳ Time /* HH:MM:SS */, HOUR, MINUTE, SECOND

↳ TimeStamp /* data, hora, fração de segundo */

→ Create Table - Empregado

```
Create Table Empregado {
  nomeinic   VARCHAR(15)   NOT NULL,
  nomeint    CHAR(1)
  nomefim    VARCHAR(15)   NOT NULL,
  numat      CHAR(4)       NOT NULL,
  dtnasc     DATE
  ender      VARCHAR(30),
  sexo       CHAR(1),
  salario    DECIMAL(10,2),
  nsuper     CHAR(4),
  ndepto     INT           NOT NULL,
  PRIMARY KEY (numat),
  FOREIGNKEY (nsuper) REFERENCES Empregado
  (numat),
  FOREIGNKEY (ndepto) REFERENCES Departamento
};
```

DDL - SQL 2

- Um esquema é uma área que agrupa os objetos de um BD, tais como tabelas, restrições, visões, domínios, autorizações, índices, etc..., pertencente a um determinado usuário/aplicação
- Caso um esquema não seja criado os objetos são colocados numa área comum para todos usuários/aplicações
- Create Schema Empresa Authorization Jsilva;
- Create Database Empresa;

→ Create Table
→ Alter Table
→ Drop Table

O Banco de Dados Empresa

- Empregado (nomeinic, nomeint, nomefim, <u>numat</u>, dtnasc, ender, sexo, salario, nsupervisor, ndepto)
- Departamento (nomedep, <u>numdep</u>, nchefe, dtinchef)
- Projeto (nomeproj, <u>numproj</u>, locproj, ndep)
- Dependente (numemp, dependent, sexo, dtnasc, tipo)
- Local Depto (<u>numdep</u>, <u>locdep</u>)
- Trabalha Em (<u>numemp</u>, <u>nproj</u>, horas)

Create Table

- Para especificar uma nova tabela e atributos
- Para cada atributo: nome, tipo de dados + domínio, restrições (atributo, chave, integridade da entidade, integridade referencial)
- NULL permitido, quando não constar nas restrições
- O usuário deve ter privilégios, além de uma área de armazenamento dos dados

1 ▸ SQL - Características e instruções

- Instruções padronizadas na mesma nomenclatura e formato para diferentes SGBD
- Migração de um SGBD para outro não requer grandes mudanças
- Portabilidade entre plataformas
- As categorias de instruções de SQL
  - DDL : Data Definition Language - Definição e Manutenção das estruturas do BD
  - DML : Data Manipulation Language - Consultas, inserções, exclusões e alterações
  - DCL : Data Control Language - Controle de privilégios
  - TCL : Transact Control Language - Controle de transações

- DDL
  - Create : criação de estruturas de objetos do BD
  - Alterate : alteração da estrutura de objetos do BD
  - Drop : Eliminação da estrutura de objetos do BD

- DML
  - Insert : inserção de dados
  - Update : alteração de dados
  - Delete : exclusão de dados
  - Select : consulta de dados

- DCL
  - Grant : Atribuição de privilégios aos usuários do BD
  - Revoke : Revogação de privilégios dos usuários do BD
  - Deny : Impedir um privilégio aos usuários do BD

- TCL
  - Commit : Confirmação das manipulações
  - Rollback : Desistência das manipulações

- Banco de Dados - SQL ★

→ SQL - A Linguagem do Banco de Dados

- Muitos SGBD's relacionais comerciais vêm com uma linguagem de alto nível declarativa
  - Especifica-se o resultado desejado e o SGBD resolve a otimização e a execução
- Início dos anos 70 a IBM desenvolveu a linguagem SEQUEL (Structured English QUEry Language) para o SYSTEM R (SGBD experimental)
  - Demonstrar a viabilidade do modelo relacional de Edgar Frank Codd
- Em 77 passou a ser chamada de SQL

→ O Padrão SQL

→ Existe um esforço contínuo por estabelecimento de um padrão SQL (ANSI)

- O primeiro padrão (1986) - SQL1 ou SQL-86
  - DDL e DML
  - 1989 extensão para integridade referencial e restrições de integridade
- O segundo padrão (1992) - SQL2 ou SQL-92
  - Facilidade de esquema e administração de dados
  - Melhoramentos no DDL e DML
  - Triggers, asserções, Ferramentas descoberta conhecimento
- Em 99 definiu-se o padrão SQL 3 ou SQL99
  - Gerenciamento de objetos (encapsulamento, subtipos, herança, polimorfismo)
- Em 2003 (SQL-2003) e 2006 (SQL-2006)
  - Introdução a características relacionadas a XML
- Em 2008 (SQL-2008)
  - Mais recursos de banco de dados de objetos